18강. 표준 클래스(API)

- 날짜와 시간 클래스
- Object 클래스
- System 클래스
- 문자열 클래스
- 포장 클래스(wrapper class)

18-1. 날짜와 시간 클래스

자바는 컴퓨터의 날짜 및 시간을 읽을 수 있도록 java.util 패키지에 Date와 Callendar 클래스를 제공하고 있다.

Date 클래스(날짜)

Date 는 날짜를 표현하는 클래스이다.

```
import java.text.SimpleDateFormat;
import java.util.Date;

public class DateEx {
    public static void main(String[] args) {
        Date now = new Date();
        System.out.println("now date: " + now);
        System.out.println("now time: " + now.getTime());

        Date past = new Date(1710756131614L);
        System.out.println("past date: " + past);

        SimpleDateFormat sdf = new SimpleDateFormat("yyyy.MM.dd HH:mm:ss");
        String strNow = sdf.format(now);
        System.out.println("format now date: " + strNow);
    }
}
```

```
now date: Mon Mar 18 19:09:17 KST 2024
now time: 1710756557651
past date: Mon Mar 18 19:02:11 KST 2024
format now date: 2024.03.18 19:09:17
```

18-1. 날짜와 시간 클래스

Calendar 클래스(날짜)

Calendar 클래스는 달력을 표현하는 추상클래스이다. 특별히 제작해서 사용할 것이 아니면 Calendar클래스의 정적 메서드인 getInstance() 메서드를 이용하여 컴퓨터에 설정된 시간대(TimeZone)를 기준으로 Calendar 하위 객체를 얻을 수 있다.

```
Calendar calendar = Calendar.getInstance();
```

Calendar 가 제공하는 날짜와 시간 정보를 얻기 위해서 get() 메서드를 이용한다. get() 메서드 매개 값으로 Calendar에 정의된 상수를 주면 상수가 의미하는 값을 리턴 한다.

```
int year   = calendar.get(Calendar.YEAR);        // 년도 리턴
int month  = calendar.get(Calendar.MONTH);       // 월 리턴
int day    = calendar.get(Calendar.DAY);         // 일 리턴
int week   = calendar.get(Calendar.WEEK);        // 요일 리턴
int amPm   = calendar.get(Calendar.AM_PM);       // 오전/오후 리턴
int hour   = calendar.get(Calendar.HOUR);        // 시간 리턴
int minute = calendar.get(Calendar.MINUTE);      // 분 리턴
int second = calendar.get(Calendar.SECOND);      // 초 리턴
```

18-1. 날짜와 시간 클래스

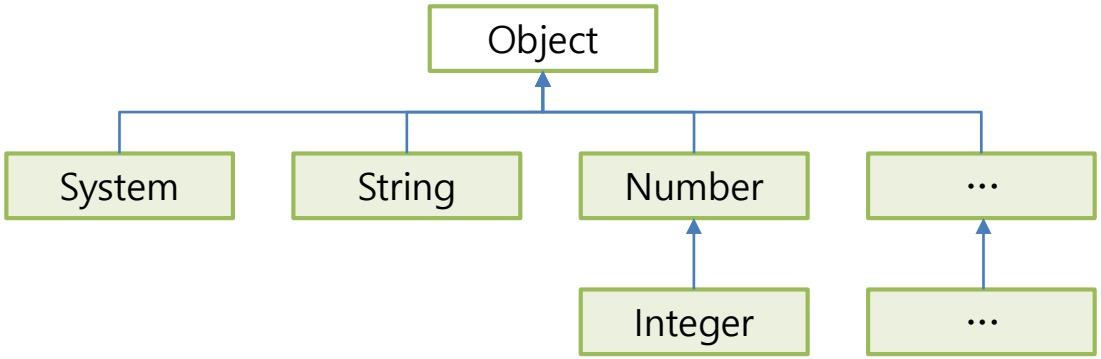
Calendar 클래스(날짜)

```java
import java.util.Calendar;
import java.util.Date;

public class CalendarEx {
    public static void main(String[] args) {
        // 현재 년도 표시
        Date now = new Date();
        Calendar calendar = Calendar.getInstance();
        calendar.setTime(now);

         int year = calendar.get(Calendar.YEAR);
         System.out.println("올해는 " + year + "년 입니다.");

         // 지정한 날의 Date형의 값 얻기
         calendar.set(2023, 3, 18, 19, 37, 30);
         Date past1 = calendar.getTime();
         System.out.println("past1: " + past1);

         calendar.set(Calendar.YEAR, 2024);
         Date past2 = calendar.getTime();
         System.out.println("past2: " + past2);
    }
}
```

```
올해는 2024년 입니다.
past1: Tue Apr 18 19:37:30 KST 2023
past2: Thu Apr 18 19:37:30 KST 2024
```

18-2. Object 클래스

클래스를 선언할 때 extends 키워드로 상속 받지 않으면 암시적으로 java.lang.Object 클래스를 상속 받게 된다. 따라서 Object 는 자바의 모든 클래스의 조상 클래스이다.



Object 가 가진 메서드는 모든 객체가 사용할 수 있다. 아래는 Object 가 가진 주요 메서드이다.

메서드	용도
boolean equals(Object obj)	객체의 번지를 비교하고 결과 리턴
int hashCode()	객체의 해시코드 리턴
String toString()	객체 문자 정보 리턴

18-2. Object 클래스

모든 클래스의 조상 Object

```
public class ObjectExam {
    static class Empty {          // extends Object 가 생략 됨

    }

    public static void main(String[] args) {
        Empty empty = new Empty();
        String s = empty.toString();
        System.out.println("s: " + s);   // s.toString()이 호출됨
    }
}
```

Object로 다형성 이용

```
public class ObjectExam2 {
    public static void main(String[] args) {
        Object obj1 = new HankookTire();
        Object obj2 = new NexenTire();
        Object obj3 = "타이어 입니다.";

        printAnything(obj2);
    }

    private static void printAnything(Object obj) {
        // 어떤 타입도 좋으니, 매개변수 1개를 받아서 화면에 표시
        System.out.println(obj.toString());
    }
}
```

18-2. Object 클래스

Object 클래스 equals 메서드 객체 동등 비교용으로 이용

equals 메서드의 매개변수 타입이 Object 이므로 자동 타입 변환에 의해 모든 객체가 매개 값으로 대입될 수 있다. equals는 객체 번지를 비교함으로 비교 연산자인 ==과 동일한 결과를 리턴 한다. 동등 비교란 객체가 서로 달라도 내부의 데이터가 같은 지 비교하는 것이다. 그래서 equals를 재 정의하여 동등 비교용으로 사용한다. 한 예로 String 클래스는 equals 메서드를 재정의하여 내부 문자열이 같은 지를 비교한다.

```java
public class Member {
    public String id;

    public Member(String id) {
        this.id = id;
    }

    // equals 재정의
    @Override
    public boolean equals(Object obj) {
        if(this == obj) {
            return true;
        }

        if(obj instanceof Member) {
            Member mbr = (Member) obj;
            if(this.id.equals(mbr.id)) {
                return true;
            }
        }
        return false;
    }
}
```

```java
public class EqualsExam {
    public static void main(String[] args) {
        Member obj1 = new Member("홍길동");
        Member obj2 = new Member("홍길순");
        Member obj3 = new Member("홍길동");

        if(obj1 == obj3) {       // 비교 연산자(주소 값)
            System.out.println("obj1과 obj2는 동등합니다.");
        } else {
            System.out.println("obj1과 obj2는 동등하지 않습니다.");
        }

        if(obj1.equals(obj2)) { // equals 재정의(객체 다름, 내부 데이터 같음)
            System.out.println("obj1과 obj2는 동등합니다.");
        } else {
            System.out.println("obj1과 obj2는 동등하지 않습니다.");
        }

        if(obj1.equals(obj3)) { // equals 재정의(객체 다름, 내부 데이터 같음)
            System.out.println("obj1과 obj2는 동등합니다.");
        } else {
            System.out.println("obj1과 obj2는 동등하지 않습니다.");
        }
    }
}
```

18-2. Object 클래스

Object 클래스 toString 기본 문자열 표현

Object의 toString() 메서드는 객체의 문자 정보를 리턴 한다. String 클래스는 저장된 문자열을 리턴하도록 toString() 메서드를 재정의하고 있다. 아래 예제는 제조회사와 운영체제 정보를 리턴하도록 toString() 메서드를 재정의 하고 있다.

```
public class SmartPhone {
    private String company;
    private String os;

    public SmartPhone(String company, String os) {
        this.company = company;
        this.os = os;
    }

    @Override
    public String toString() {
        return company + ", " + os; // 제조사 + 운영체제 리턴
    }
}
public class ToStringExam {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        SmartPhone sp = new SmartPhone("삼성전자", "안드로이드");
        // 재정의한 toString() 호출
        System.out.println(sp.toString());
        // System.out.println() 메서드의 매개값이 객체이면 toString() 호출해서 리턴
        System.out.println(sp);
    }
}
```

toString() 을 오버라이드

18-3. System 클래스

- 자바는 자바 가상 머신(JVM) 위에서 실행되기 때문에 운영체제의 기능에 직접 접근하기가 어려우나 java.lang 패키지에 속하는 System클래스를 이용하면 운영체제의 일부 기능을 이용할 수 있다.
- System클래스의 정적(static) 필드와 메서드를 이용하면 프로그램 종료, 키보드 입력, 콘솔 출력, 현재 시간 읽기, 시스템 프로퍼티 읽기 등이 가능하다.

정적 멤버		용도
필드	out	콘솔(모니터)에 문자 출력
	err	콘솔(모니터)에 에러 내용 출력
	in	키보드 입력
메서드	exit(int status)	프로세스 종료
	currentTimeMillis()	현재 시간 밀리초 단위의 long 값 리턴
	nanoTime()	현재 시간 나노초 단위의 long 값 리턴
	getProperty	운영체제와 사용자 정보 제공
	getenv()	운영체제의 환경 변수 정보 제공

18-3. System 클래스

콘솔 출력

out 과 err 필드를 이용하면 콘솔에 원하는 문자열을 출력할 수 있다. 차이점은 err은 콘솔 종류에 따라 에러 내용이 빨간색으로 출력된다.

```
public class ErrExam {
    public static void main(String[] args) {
        try {
            int iValue = Integer.parseInt("loo");
        } catch(NumberFormatException e) {
            System.out.println("에러 내용: " + e.getMessage());

            System.err.println("에러 내용: " + e.getMessage());
        }
    }
}
```

```
에러 내용: For input string: "loo"
에러 내용: For input string: "loo"
```

18-3. System 클래스

키보드 입력

자바는 키보드 입력을 받기 위해 System 클래스의 in 를 제공한다.

```
int keycode = System.in.read();
```

```
public class KeyInExam {
    public static void main(String[] args) throws Exception {
        int keyCode = 0;

        while(true) {
            // Enter 키(13, 10)를 읽지 않았을 경우에만 실행
            if( keyCode != 13 && keyCode != 10 ) {
                if(keyCode == 49) {                // 숫자 1을 읽었을 경우
                    System.out.println("입력 받은 KeyCode 값: " + keyCode);
                    break;
                }
            }

            keyCode = System.in.read();
        }
        System.out.println("프로그램 종료");
    }
}
```

18-3. System 클래스

프로세스 종료

자바는 System.exit() 메서드로 프로세스를 강제 종료 시킬 수 있다. exit() 메서드는 매개 값이 필요한데 정상 종료(0), 비정상 종료(1 또는 -1) 값으로 주는 것이 관례이다.

```
System.exit(int status);
```

```
public class ExitExam {
    public static void main(String[] args) {
        for( int i = 0; i < 10; i++ ) {
            System.out.println("i: " + i);
            if( i == 5 ) {
                System.out.println("JVM 프로세스 강제 종료");
                System.exit(0);
            }
        }
    }
}
```

18-4. 문자열 클래스

자바에서 문자열 관련 주요 클래스는 아래와 같다.

메서드	용도
String	문자열 저장 및 조작 시 사용
StringBuilder	문자열 수정 등 효율적으로 문자열 조작 시 사용
StringTokenizer	구분자로 연결된 문자열 분리 시 사용

18-4. 문자열 클래스 - String

String 클래스

- String 클래스는 문자열을 저장하거나 조작할 때 사용한다. String 은 내부 문자열을 수정할 수 없다. 문자열 결합을 하면 내부 문자열을 변경하는 것처럼 보이지만 신규 String 객체를 생성하는 것이다. 빈번한 내부 문자열 수정이 필요하다면 String 클래스의 사용은 효율적이지 않다.

리턴 타입	주요 메서드(매개변수)	설명
String	concat(문자열)	문자열 연결
String	substring(시작 위치, 끝 위치)	문자열 자르기
int	length()	문자열 길이
String	toUpperCase(문자열)	대문자로 만들기
String	toLowerCase(문자열)	소문자로 만들기
char	charAt(위치)	특정 위치의 글자 찾기
int	indexOf(문자 | 문자열)	특정 문자열 위치
boolean	equals(문자열)	문자열 비교
String	trim()	문자열 공백 제거
String	replace(원본 문자|문자열, 대체 문자|문자열)	특정 문자 교체
String	replaceAll(원본 문자열, 대체 문자열)	특정 문자열 교체

18-4. 문자열 클래스 - String

문자열 연결

```
String str1 = "Hello";
String str2 = " Java";

System.out.println(str1 + str2);
System.out.println(str1.concat(str2));
```

```
Hello Java
Hello Java
```

문자열 자르기

```
String str3 = "Hello";

System.out.println(str3.substring(0, 2));
```

```
He
```

문자열 길이

```
String str4 = "Hello Java";

System.out.println(str4.length());
System.out.println(str4.isEmpty());
```

```
10
false
```

isEmpty(): 길이가 0인지 검사

18-4. 문자열 클래스 - String

문자열 변환(대문자/소문자/공백 제거/교체)

```
String str5 = "Java and Python";

System.out.println(str5.toUpperCase());          // 대문자로        JAVA AND PYTHON
System.out.println(str5.toLowerCase());          // 소문자로        java and python
System.out.println(str5.trim());                 // 좌우 공백 제거  Java and Python
System.out.println(str5.replace("and", "or"));   // 교체            Java or Python
System.out.println(str5.replace('a', 'A'));      // 교체            JAvA And Python
System.out.println(str5.replaceAll("and", "or"); // 교체            Java or Python
```

문자열 찾기(검색)

```
String str6 = "Java and Python";

System.out.println(str6.charAt(10));        // 특정위치 글자      Y
System.out.println(str6.indexOf("and"));    // 특정문자열 위치    5
System.out.println(str6.contains("Java"));  // 포함관계           true

System.out.println();
```

contains(): 포함 관계

18-4. 문자열 클래스 - String

문자열 비교

```
String str7 = "Java";
String str8 = "java";

if(str7.equals(str8)) {
    System.out.println("str7과 str8은 같다");
}

if(str7.equalsIgnoreCase(str8)) {
    System.out.println("str7과 str8은 대소문자 무시하면 같다");
}
```

```
str7과 str8은 대소문자 무시하면 같다
```

18-4. 문자열 클래스 – StringBuilder

StringBuilder 클래스

- StringBuilder 클래스는 내부 버퍼에 문자열을 저장해두고 조작(추가, 변경, 삭제 등)하도록 설계되어 있어 String(불변) 객체처럼 새로운 문자열을 만들지 않고 조작할 수 있어 메모리 측면에서 좀더 효율적이다.
- StringBuilder와 StringBuffer는 동일한 메서드를 제공 유사하나 아래와 같은 차이점이 있다.

StringBuilder

- StringBuffer와 유사하지만 동기화되어 있지 않습니다. 단일 스레드 환경에서 더 빠르게 동작합니다.

StringBuffer

- StringBuffer는 동기화되어 있어 멀티스레드 환경에서 안전하지만, 성능이 다소 저하될 수 있습니다.

리턴 타입	주요 메서드(매개변수)	설명
StringBuilder	append(기본값|문자열)	문자열 끝에 추가
StringBuilder	insert(위치, 기본값|문자열)	문자열 지정 위치에 추가
StringBuilder	delete(시작 위치, 끝 위치)	문자열 일부 삭제
StringBuilder	replace(시작 위치, 끝 위치, 문자열)	문자열 일부 대체
String	toString()	완성된 문자열 리턴

18-4. 문자열 클래스 - StringBuilder

메서드 사용

```
StringBuilder sb = new StringBuilder();

sb.append("Java Programming");
System.out.println(sb);

sb.insert(5, "and Python Programming");
System.out.println(sb);

sb.delete(17, 28);
System.out.println(sb);

sb.replace(9, 15, "JSP");
System.out.println(sb.toString());
```

```
Java Programming
Java and Python ProgrammingProgramming
Java and Python Programming
Java and JSP Programming
```

메서드 체이닝 패턴 – toString() 제외한 메서드가 자기 자신 참조를 리턴(StringBuilder)하여 연속해서 메서드를 호출하는 기법

```
StringBuilder sb2 = new StringBuilder();
sb2.append("DEF").insert(0, "ABC").delete(3, 4);
System.out.println(sb2);
```

```
ABCEF
```

18-5. 포장 클래스(wrapper class)

- 자바는 기본 타입의 값을 갖는 객체를 생성할 수 있다. 즉 기본 데이터 형을 객체로서 사용한다는 의미이다. 이런 객체를 포장(wrapper) 객체라고 한다.

기본 타입(primitive type)	래퍼 타입(wrapper class)
byte	java.lang.Byte
short	java.lang.Short
int	java.lang.Integer
long	java.lang.Long
float	java.lang.Float
double	java.lang.Double
char	java.lang.Character
boolean	java.lang.Boolean

18-5. 포장 클래스(wrapper class)

기본 타입 -> 오브젝트 변환 – 포장 객체는 포장하고 있는 기본 타입의 값을 변경할 수 없고, 단지 객체로 생성하는 데 목적이 있다.

```
public class WrapperEx {
    public static void main(String[] args) {
        int iVal1 = 49;
        Integer obj1 = Integer.valueOf(iVal1);
        int iVal2 = obj1.intValue();
        System.out.println(iVal2);
    }
}
```

박싱(boxing)과 언박싱(unboxing) -> 기본 타입을 포장 객체로 만드는 과정을 박싱(boxing), 반대로 포장 객체에서 기본 타입의 값을 얻어내는 과정을 언박싱(unboxing) 이라고 한다.

```
public class WrapperEx {
    public static void main(String[] args) {
        int iVal1 = 49;

        Integer obj = iVal1;                          // Boxing
        System.out.println("값: " + obj.intValue()); // Integer 객체 내부의 int 값 리턴

        int value = obj;                              // UnBoxing
        System.out.println("값: " + value);

        int result = obj + iVal1;                     // 연산 시 UnBoxing
        System.out.println("값: " + result);
    }
}
```

18. 요점 정리

날짜와 시간 클래스

- Java에서 날짜 정보는 기본적으로 java.util.Date 를 사용
- 필요에 의해 long 형, 6개의 int, String 형으로 변환하여 사용
- '년 월 일 시 분 초' 의 6개의 int 값으로 부터 Date 인스턴스를 얻기 위해 Calendar 클래스를 사용
- Date 인스턴스의 내용을 임의의 형식으로 문자열 표시하기 위해서는 SimpleDateFormat 클래스를 사용

Object 클래스

- Java에서, 모든 클래스는 Object 클래스의 자식이다.
- 모든 인스턴스는 Object 형 변수에 대입할 수 있다.
- 모든 클래스는 Object 로부터 toString() 이나 equals() 를 상속 받고 있다.
- 자신이 작성한 클래스를 이용해, 문자열 표현이나 같은지 아닌지 비교를 하기 위해서는, toString() 이나 equals()를 오버라이드 해야한다.

포장 클래스(wrapper class)

- API에는 여러 예외적 상황을 표현하는 예외클래스가 있다.
- 예외 클래스는 Throwable을 상속 받아 Error 계열, Exception 계열, RuntimeException 계열로 나뉜다.
- 예외 클래스를 상속하여 사용자 정의 예외 클래스를 정의 할 수 있다.

18. 예제

[예제 1] (java_22_99_ex1_standardclassex : KeyInExam)

아래의 조건을 만족하는 프로그램을 작성하시오.

1. System 클래스의 in 필드를 사용하여 keyCode를 입력 받으시오.
2. 숫자 키 1(49)과 2(50)를 입력함에 따라 speed 변수 값을 증감하고, 숫자 키 3(51)을 입력하면 "프로그램 종료" 라는 메시지를 출력하고 프로그램을 종료 하시오.

[예제 2] (java_22_99_ex2_Account)

계좌번호를 표시하는 String 형 필드 accountNumber 와 잔액을 표시하는 int 형 필드 balance 를 가지는 은행계좌 클래스를 작성하시오. 그리고, 이 클래스에 메소드 선언을 추가해, 다음의 1.과 2.의 조건을 만족하도록 수정 하시오.

1. 계좌번호 4649, 잔액 1592원 Account 인스턴스를 변수 a 에 생성하고 **System.out.println(a);** 를 실행하면, 화면에 **1592원 (계좌번호=4649))** 라고 표시
2. 계좌번호가 같으면 값이 같은 것으로 판단할 것. 단, " 4649" 처럼 계좌번호의 앞에 공백이 있으면, 공백을 무시하고 비교할 것 ("4649"와 " 4649 "는 같은 것으로 본다)
   (힌트 : java.lang.String 클래스의 trim() 메소드를 이용)

18. 예제

[예제 3] (java_22_99_ex3_datecalendar)

main() 메서드만 가지는 클래스 DateCalendar 을 정의하고, 이하의 순서로 **현재의 100일 후의 날짜** 를 "**서기 2024년 09월 24일**" 와 같은 형식으로 표시하시오.

- 현재의 날짜를 Date 형으로 얻는다.
- 얻은 날짜 정보를 Calendar 에 설정한다.
- Calendar 에서 **일(day)** 값을 얻는다.
- 얻은 값에 100을 더한 값을 Calendar 의 **일** 에 설정한다.
- Calendar 의 날짜 정보를 Date 형으로 변환한다.
- SimpleDateFormat 을 이용하여 Date 인스턴스의 내용을 표시한다.